

Teste de aplicações

Introdução a Testes de Software

Os testes de software são um processo essencial no desenvolvimento de sistemas, garantindo que os programas funcionem corretamente, atendam aos requisitos e sejam confiáveis para os usuários. Eles ajudam a identificar defeitos antes que o software seja implantado, reduzindo custos e melhorando a qualidade.

Teste de software é a atividade de verificar e validar se um software atende às especificações desejadas e se comporta conforme esperado. O objetivo é encontrar defeitos (bugs) e garantir que o sistema opere de forma correta, eficiente e segura.

Princípios dos Testes de Software

Os testes de software seguem princípios fundamentais que ajudam a torná-los mais eficazes e estratégicos. O **ISTQB** (International Software Testing Qualifications Board) definiu sete princípios essenciais para guiar a prática de testes.

1. Teste mostra a presença de defeitos, não sua ausência

Os testes ajudam a encontrar falhas no software, mas não garantem que ele esteja 100% livre de erros. Mesmo que um software passe em todos os testes, isso não significa que ele seja perfeito ou imune a problemas.

Exemplo: Um site pode funcionar corretamente em testes, mas um usuário pode encontrar um erro em um cenário inesperado.

2. Teste exaustivo é impossível

Testar todas as combinações possíveis de entrada e saída é inviável na maioria dos casos. Em vez disso, os testes devem ser planejados de forma estratégica, focando nas áreas mais críticas.

Exemplo: Um aplicativo com múltiplas funcionalidades teria um número infinito de combinações de testes. Técnicas como particionamento de equivalência ajudam a reduzir o esforço de teste.

3. Teste precoce economiza tempo e dinheiro

Quanto antes os testes forem realizados no ciclo de desenvolvimento, mais barato e fácil será corrigir os erros. O custo de um defeito cresce conforme

ele avança pelas fases do projeto.

Exemplo: Se um erro for encontrado na fase de requisitos, corrigi-lo será muito mais barato do que após a implementação ou a produção.

4. Agrupamento de defeitos (Regra 80/20)

Geralmente, a maioria dos defeitos está concentrada em um pequeno número de módulos ou funcionalidades. Isso é conhecido como o princípio de Pareto (80% dos defeitos vêm de 20% do sistema).

Exemplo: Em um e-commerce, pode ser que a área de pagamentos tenha mais falhas do que a página inicial, exigindo mais atenção nos testes.

5. O paradoxo do pesticida

Repetir sempre os mesmos testes com os mesmos casos pode fazer com que erros novos passem despercebidos. É importante revisar e atualizar os testes regularmente.

Solução: Criar cenários de teste.

Usar testes exploratórios para encontrar erros inesperados.

Exemplo: Um sistema pode passar sempre nos testes automatizados, mas um novo comportamento inesperado pode surgir sem ser detectado.

6. Os testes dependem do contexto

O tipo e a abordagem dos testes variam de acordo com o software. Sistemas críticos (como bancos e hospitais) exigem testes mais rigorosos do que um site de entretenimento.

Exemplo: Um aplicativo bancário exige testes rigorosos de segurança e precisão.

Um jogo pode focar mais em testes de experiência do usuário e gráficos.

7. “Ausência de erros” não significa software útil

Um software pode ser tecnicamente perfeito (sem bugs), mas ainda assim falhar em atender às necessidades do usuário. O objetivo final do software é fornecer valor ao usuário.

Exemplo: Um aplicativo pode funcionar sem falhas técnicas, mas ser difícil de usar e não atender às expectativas dos clientes.

Importância dos Testes de Software

1. Melhoria da Qualidade

- Identificam erros e inconsistências antes da entrega.
- Garantem que o software atenda às expectativas do usuário.

2. Redução de Custos

- Corrigir defeitos na fase de desenvolvimento é mais barato do que após a implantação.

- Evita gastos desnecessários com manutenção corretiva.

3. Segurança e Confiabilidade

- Identificam vulnerabilidades e falhas críticas.

- Previnem ataques cibernéticos e problemas de privacidade.

4. Satisfação do Usuário

- Proporcionam uma experiência fluida e sem falhas.

- Aumentam a confiança do cliente no software.

5. Cumprimento de Requisitos

- Asseguram que o software atende às especificações funcionais e não funcionais.

- Evitam retrabalho e mudanças inesperadas no projeto.

6. Facilidade de Manutenção

- Um código bem testado é mais fácil de modificar e expandir.

- Reduz a ocorrência de erros futuros ao introduzir novas funcionalidades.

Objetivos dos Testes de Software

1. Detectar Defeitos o Mais Cedo Possível

- Identificar falhas antes que cheguem ao usuário final.

2. Garantir Conformidade com os Requisitos

- Assegurar que o software funciona conforme especificado.

3. Validar a Experiência do Usuário

- Testar a usabilidade e garantir uma interface amigável.

4. Avaliar Desempenho e Estabilidade

- Testar a capacidade do sistema sob diferentes cargas de trabalho.

5. Automatizar Processos para Eficiência

- Reduzir tempo e esforço manual na execução dos testes.

6. Assegurar a Integração entre Componentes

- Testar a comunicação entre diferentes partes do sistema.

7. Garantir a Segurança da Informação

- Identificar e corrigir vulnerabilidades que possam comprometer dados.

Tipos de Testes

Os testes podem ser classificados de diversas formas, dependendo do objetivo e da abordagem utilizada:

1. Testes Funcionais

Verificam se o software atende aos requisitos funcionais.

Exemplos:

- Teste de Unidade
- Teste de Integração
- Teste de Sistema
- Teste de Aceitação

2. Testes Não Funcionais

Avaliam características como desempenho, segurança e usabilidade.

Exemplos:

- Teste de Performance
- Teste de Segurança
- Teste de Usabilidade

3. Testes Automatizados vs. Manuais

- **Teste Manual:** Realizado por um testador que executa os casos de teste sem o uso de ferramentas automatizadas.
- **Teste Automatizado:** Usa scripts e ferramentas para executar testes automaticamente, aumentando a eficiência e a repetibilidade.

Ferramentas Populares de Teste

- Selenium (automação de testes web)
- JUnit (teste unitário para Java)
- Postman (testes de API)
- JMeter (teste de desempenho)

O que são Cenários e Casos de Teste?

a. **Cenário de Teste** → Um conjunto de condições que descreve uma situação real em que o software pode ser usado. Define **o que testar**, sem entrar em detalhes técnicos.

b. **Caso de Teste** → Um conjunto de entradas, ações e resultados esperados para validar um determinado cenário. Especifica **como testar** o sistema.

Diferença principal:

- O **cenário** é mais genérico, descrevendo um contexto amplo.
- O **caso de teste** é detalhado, incluindo etapas específicas, entradas e saídas esperadas.

Estrutura de um Caso de Teste

Um **caso de teste** bem elaborado geralmente contém:

1. **ID do Teste** → Identificação única (ex: CT-001).
2. **Título** → Nome que resume o objetivo do teste.
3. **Descrição** → O que será testado e qual a funcionalidade envolvida.
4. **Pré-condições** → O que precisa estar configurado antes do teste (ex: usuário logado).
5. **Passos** → Sequência de ações a serem executadas.
6. **Dados de Teste** → Valores de entrada usados no teste.
7. **Resultado Esperado** → O que deve acontecer se o sistema estiver correto.
8. **Resultado Obtido** → O que realmente aconteceu (preenchido após execução).
9. **Status** → Indica se o teste **Passou**, **Falhou** ou está **bloqueado**.

Criando um Cenário de Teste

Para criar um cenário de teste eficiente, siga estes passos:

Passo 1: Identificar funcionalidades críticas

- Quais áreas do sistema são mais importantes?
- Onde há maior risco de falhas?

Exemplo: Testar **cadastro de usuários, processo de pagamento, envio de e-mails**.

Passo 2: Definir situações reais de uso

- O que um usuário comum faria no sistema?
- Existem diferentes tipos de usuários com comportamentos distintos?

Exemplo: **Cenário**: Cliente realiza uma compra online e recebe um e-mail de confirmação.

Passo 3: Criar casos de teste detalhados

- Para cada cenário, criar casos de teste detalhados, cobrindo entradas válidas e inválidas.

Exemplo de casos de teste para o cenário "Realizar compra online":

1. Compra com cartão válido
2. Compra com cartão inválido
3. Compra com frete grátis aplicado
4. Compra cancelada antes da finalização

Boas Práticas na Criação de Testes

1. Testes bem escritos evitam ambiguidades

- Escreva testes com descrições claras, que deixem evidente o que está sendo testado, qual o comportamento esperado e em que condições. Isso

ajuda a equipe a entender rapidamente a intenção do teste e reduz erros de interpretação.

2. Use uma linguagem simples e direta

- Evite jargões técnicos desnecessários. Escreva de forma que tanto desenvolvedores quanto testadores consigam entender facilmente o que o teste faz, mesmo que não sejam especialistas na funcionalidade testada.

3. Cubra cenários positivos e negativos

- Cenários positivos: verifique se o sistema funciona como esperado em situações normais.
- Cenários negativos: teste entradas inválidas, falhas de sistema ou comportamentos inesperados para garantir que o sistema lide bem com erros.

4. Priorize os testes mais críticos primeiros

- Comece testando as funcionalidades mais importantes do sistema, aquelas que impactam diretamente o usuário ou a operação do negócio. Isso ajuda a identificar problemas relevantes logo no início.

5. Automatize quando possível

- Testes que são executados com frequência (como testes de regressão) devem ser automatizados. Isso economiza tempo, reduz o esforço manual e aumenta a confiabilidade dos testes.

GitHub da Disciplina

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com-Metodos-Ageis>

Conteúdo Bônus

Uma série que mostra de uma forma bem-humorada os bastidores do desenvolvimento de software com bugs inesperados, deploys desastrosos e testes malfeitos é a série Silicon Valley de 2014, nela podemos ver como é importante ter testes bem executados e de boa confiabilidade.

Referências Bibliográficas

BURNSTEIN, Ilene. Practical Software Testing: A Process-Oriented Approach. New York: Springer, 2003.

FEWSTER, Mark; GRAHAM, Dorothy. Software Test Automation. Boston: Addison-Wesley, 1999.

GLENFORD, Myers. The Art of Software Testing. Hoboken: John Wiley & Sons, 2011.

Diferentes tipos de testes de software. Sten Pittet. Disponível em <https://www.atlassian.com/br/continuous-delivery/software-testing/types-of-software-testing>. Acesso em 10 de abril de 2025.

Quais os tipos de teste que devo fazer em aplicações móveis? Elisabeth Mamede, novembro de 2018. Disponível em <https://medium.com/@elisabethmamede/quais-os-tipos-de-teste-que-devo-fazer-em-aplicações-móveis-abbfb71c9aa1>. Acesso em 10 de abril de 2025.

O que é teste de software? IBM. Disponível em <https://www.ibm.com/br-pt/topics/software-testing>. Acesso em 10 de abril de 2025.

Atividade Prática 7 – Teste de aplicações

Título da Prática: Testes de aplicações

Objetivos: Trabalhar com cenários de testes de aplicações

Materiais, Métodos e Ferramentas: Para realizar esta prática vamos ler atentamente o conteúdo da unidade

Atividade Prática

1. Em uma empresa de desenvolvimento, um novo estagiário questionou por que é necessário investir tempo e recursos em testar o software, uma vez que os desenvolvedores já verificam o código que produzem. O gerente aproveitou para introduzir a disciplina de testes de software.

Atividade:

Explique com suas palavras o que são testes de software e por que eles são considerados uma atividade essencial no processo de desenvolvimento.

2. Durante um curso sobre qualidade de software, um instrutor apresentou os princípios fundamentais de testes, destacando que nem sempre é possível testar tudo. Um aluno ficou surpreso com essa afirmação.

Atividade:

Explique o princípio do “teste exaustivo é impossível” e justifique por que ele é importante para orientar as atividades de teste.

3. Em uma reunião, o gerente de projeto afirmou que testar um software não serve apenas para encontrar erros, mas também para alcançar outros objetivos importantes. Alguns membros da equipe não entenderam completamente esse ponto.

Atividade:

Explique dois objetivos dos testes de software além da detecção de defeitos e porque são importantes para o sucesso de um projeto.

4. Uma equipe está definindo a estratégia de testes para um novo sistema bancário. Um dos analistas sugeriu que eles realizem tanto testes funcionais quanto testes não funcionais.

Atividade:

Explique a diferença entre testes funcionais e não funcionais, dando um exemplo de cada tipo que seria relevante para o sistema bancário.

5. Uma empresa está desenvolvendo um aplicativo de delivery e contratou um analista de testes para criar cenários e casos de teste. O analista explicou que é necessário compreender bem o funcionamento do sistema para criar artefatos que representem situações reais de uso.

Atividade:

Explique a diferença entre cenário de teste e caso de teste, e a importância de ambos para garantir a qualidade do aplicativo.

Gabarito Atividade Prática

1. Testes de software são atividades sistemáticas realizadas para verificar se um sistema ou aplicação atende aos requisitos especificados e se funciona corretamente. Eles são essenciais para identificar falhas antes da entrega ao cliente, garantindo a qualidade, confiabilidade e segurança do produto. Além disso, ajudam a reduzir custos com manutenção e aumentam a satisfação do usuário final.

2. O princípio do “teste exaustivo é impossível” significa que não é viável testar todas as combinações possíveis de entradas e cenários em um software, especialmente em sistemas complexos. Esse princípio é importante porque orienta os profissionais a focarem em testes que maximizem a detecção de falhas com o menor esforço, utilizando técnicas de seleção de casos de teste eficientes e baseadas em risco.

3. Além de detectar defeitos, os testes de software também têm o objetivo de

demonstrar a conformidade com os requisitos e aumentar a confiança na qualidade do sistema. Demonstrar conformidade garante que o sistema atenda às necessidades do cliente e requisitos legais. Já aumentar a confiança permite que a equipe e os stakeholders tenham segurança para liberar o software para produção, sabendo que ele foi adequadamente validado.

4. Testes funcionais verificam se o sistema realiza corretamente as funções especificadas, como processar transações financeiras ou autenticar usuários. Já os testes não funcionais avaliam aspectos como desempenho, segurança e usabilidade. No sistema bancário, um exemplo de teste funcional seria validar se a transferência entre contas ocorre corretamente. Um exemplo de teste não funcional seria verificar se o sistema suporta mil acessos simultâneos sem degradar a performance.

5. Um **cenário de teste** descreve uma situação ou fluxo geral de uso que será avaliado, enquanto um **caso de teste** é um conjunto específico de condições, entradas e ações, com resultados esperados, derivados desse cenário. Os cenários ajudam a garantir que as principais funcionalidades e jornadas do usuário sejam contempladas, enquanto os casos detalham como cada situação será verificada. No aplicativo de delivery, por exemplo, um cenário seria “realizar um pedido de comida”, e um caso de teste seria “realizar um pedido com pagamento via cartão de crédito e entrega agendada para o dia seguinte”.